MediFlow — Project Brief for Students

Smart Clinic & Appointment Optimization System · Oct 21, 2026 · @Ggg

1. Project overview

You will build **MediFlow**, a multilingual (English, Arabic with full RTL, French) web platform that lets clinics manage appointments, doctors, patients, queues and operational analytics, and that suggests better schedules The target is a product that looks and behaves like a sellable healthcare SaaS, not a CRUD exercise. The UI mockups (dashboard, appointments, create-appointment wizard, calendar, patients) are the visual reference; your front end must match their layout, colour system and quality.

to reduce waiting time.

What you will practise

- Designing and building a modular back end (NestJS) with relational and document databases.
- Real-time features (WebSockets) for queue, chat and notifications.
- Caching, rate limiting and background jobs with Redis.
- A professional React front end with a design system, i18n and RTL.
- Testing, CI/CD, security and documentation like a real product team.

Design tokens (from the Figma brief)

Token	Value
Primary (deep teal)	`#0F766E`
Secondary (navy)	`#15283F`
Background / Surface	`#F7F9FC` / `#FFFFFF`
Text primary / secondary	`#172B4D` / `#697586`
Border	`#E5EAF0`
Success / Warning / Error	`#16A34A` / `#F59E0B` / `#DC2626`
Accent (soft cyan)	`#DDF6F3`
Font / radius	Inter / 10 to 16 px

2. Roles, scope and out of scope

User roles (enforced by RBAC on the API and hidden in the UI):

Role	Can do
Super Admin	Manage clinics, users, global settings, audit log
Clinic Admin / Manager	Everything inside one clinic: staff, settings, analytics, reports
Doctor	See own schedule and patients, start and complete consultations, manage own availability
Receptionist	Book, reschedule, cancel, check in patients, manage the queue
Patient (optional portal)	View own appointments, book, cancel, receive reminders

In scope: every screen in the Figma brief (dashboard, appointments, create flow, smart scheduling, waiting list, patients, doctors and staff, analytics, notifications, settings), plus chat and real-time notifications.

Out of scope: real medical records (diagnoses, prescriptions), billing and insurance claims, real SMS or WhatsApp delivery (use a mock provider behind an interface), and machine-learning models. Smart scheduling uses a deterministic heuristic.

Data rule: use fictional data only. Never put real patient data in the repo, seeds or screenshots.

3. Technology stack

The stack is mandatory unless your supervisor approves a change. Pin versions in `package.json` and document them in the README.

Layer	Technology	Used for
Front end	React 18 + TypeScript, Vite	SPA, routing with React Router
UI and state	Tailwind CSS (or MUI with a custom theme), shadcn/ui or Radix, TanStack Query, Zustand	Design system, server state, client state
Forms and validation	React Hook Form + Zod	Typed forms shared with API schemas

Layer	Technology	Used for
Charts and calendar	Recharts or ECharts, FullCalendar or custom grid, dnd-kit	Analytics, calendar, drag and drop scheduling
i18n	i18next + react-i18next	English, Arabic (RTL), French
Real time (client)	socket.io-client	Queue, chat, notifications
API	NestJS (Node 20 LTS) + TypeScript	REST API, WebSocket gateways, modules, guards
ORM	TypeORM or Prisma (pick one for the team)	PostgreSQL access and migrations
Primary database	PostgreSQL 16	Users, patients, doctors, appointments, schedules, audit
Cache and queues	Redis 7 + BullMQ	Caching, sessions and token blacklist, rate limiting, slot locks, background jobs, Socket.IO adapter
Document database	MongoDB 7 (Mongoose)	Chat messages, notifications feed, delivery logs
API docs	Swagger / OpenAPI via `@nestjs/swagger`	Contract between front and back end
Auth	JWT access + refresh tokens, bcrypt or argon2	Authentication and RBAC
Testing	Jest, Supertest, React Testing Library, Playwright	Unit, integration, end to end
DevOps	Docker + Docker Compose, GitHub Actions, ESLint, Prettier, Husky	Local environment, CI, code quality
Observability	Pino logs, `/health` endpoint, optional Prometheus + Grafana	Monitoring

Why three data stores? PostgreSQL holds relational, transactional data that needs constraints and joins. Redis holds data that is fast, temporary or shared between instances. MongoDB holds high-volume, schema-flexible, append-heavy data such as messages and notifications.

4. Architecture and repository layout

Build a **modular monolith** in NestJS: one deployable API, with strictly separated modules that could become microservices later. The chat and notification module is the exception: it runs as a second NestJS app (`realtime-service`) backed by MongoDB, to practise service-to-service communication.

Request path: React SPA, then REST (`/api/v1`) and Socket.IO, then NestJS guards (JWT, roles, clinic scope), then services, then PostgreSQL. Redis sits beside the API for cache, locks and queues. The realtime service reads and writes MongoDB and receives domain events (for example `appointment.created`) from the API through a Redis pub/sub channel or BullMQ queue.

Who owns which data

Store	Owns
PostgreSQL	clinics, users, roles, patients, doctors, specialties, working hours, leaves, appointments, queue entries, settings, audit log
Redis	refresh-token blacklist, rate-limit counters, cached dashboards and slot lists, temporary slot locks (TTL 5 min), BullMQ jobs, Socket.IO adapter and presence
MongoDB	chat conversations and messages, notifications feed, notification delivery log, read receipts

Repository layout (monorepo)

```
mediflow/
  apps/
    web/                # React + Vite
    api/                # NestJS core API (PostgreSQL, Redis)
    realtime-service/   # NestJS chat + notifications (MongoDB, Redis)
  packages/
    shared/             # shared types, Zod schemas, constants
  infra/
    docker-compose.yml  # postgres, redis, mongo, api, realtime, web
  docs/                 # ADRs, API docs, ERD, user manual
  .github/workflows/    # CI pipelines
```

API modules: auth, users, clinics, patients, doctors, specialties, availability, appointments, scheduling-optimizer, queue, analytics, reports, settings, audit, notifications-gateway (client of realtime-service).

API conventions: versioned REST, DTO validation with `class-validator`, a global exception filter returning `{ code, message, details }`, pagination via `page`, `limit`, `sort`, and ISO 8601 UTC timestamps (clinic time zone applied in the UI).

5. Working method and Definition of Done

Team and process. Teams of 3 to 4. Work in 2-week sprints (6 sprints). Each sprint starts with planning from the backlog below and ends with a demo and a retrospective. Track tickets in GitHub Projects or Jira, one ticket per task ID (for example `APP-3`).

Git workflow

- Branches: `main` (protected, always deployable), `develop`, and `feature/<task-id>-short-name`.
- Conventional commits (`feat:`, `fix:`, `test:`, `docs:`).
- Every merge needs a pull request, one reviewer, and a green CI run.

Definition of Done (applies to every task)

- [ ] All acceptance criteria of the task are met and demonstrated.
- [ ] Unit tests for new logic; integration test for each new endpoint.
- [ ] Input validated on client and server; errors handled with clear messages.
- [ ] Strings are translated in EN, AR and FR; layout checked in RTL.
- [ ] Keyboard accessible, with visible focus states and sufficient contrast.
- [ ] Swagger updated; migration files committed; no secrets in the repo.
- [ ] Lint, type-check and tests pass in CI; PR reviewed and merged.

Task ID prefixes: `FND` foundation, `AUTH` authentication, `I18N` languages, `PAT` patients, `DOC` doctors, `APP` appointments, `SMT` smart scheduling, `QUE` queue, `RT` realtime, `ANA` analytics, `SET` settings, `UI` design system, `QA` testing and DevOps.

6. Backlog, part 1: foundation, authentication, languages

Each task lists subtasks (the work) and acceptance criteria (AC, how it is verified). A task is accepted only when every AC passes in a demo.

Epic 0 — Foundation (Sprint 1)

FND-1 Monorepo and local environment

Subtasks:

- Create monorepo (pnpm workspaces), `apps/web`, `apps/api`, `apps/realtime-service`, `packages/shared`.
- Write `docker-compose.yml` for PostgreSQL, Redis, MongoDB and the apps.
- Add ESLint, Prettier, Husky pre-commit, commit lint, `.env.example`.

Acceptance criteria:

- `docker compose up` starts all services from a clean clone in under 5 minutes.
- `GET /health` on both back ends returns the status of each connected database.
- README explains setup in 10 steps or fewer.

FND-2 Database schema and migrations

Subtasks:

- Design the ERD (clinics, users, roles, patients, doctors, specialties, working_hours, leaves, appointments, queue_entries, settings, audit_logs).
- Write migrations and seed script with fictional data (3 clinics, 10 doctors, 200 patients, 600 appointments).
- Add indexes on `appointments(doctor_id, starts_at)` and `appointments(patient_id)`.

Acceptance criteria:

- ERD exported to `docs/` and matches the migrations.
- `migrate` and `seed` run idempotently on an empty database.
- Foreign keys and unique constraints prevent orphan rows and duplicate patient IDs per clinic.

FND-3 API skeleton and CI

Subtasks:

- Global validation pipe, exception filter, logging (Pino), config module, Swagger at `/docs`.
- GitHub Actions: install, lint, type-check, test, build on every PR.

Acceptance criteria:

- Invalid payloads return HTTP 400 with field-level errors.
- CI fails on lint or test failure and blocks the merge.

Epic 1 — Authentication and access control (Sprint 1–2)

AUTH-1 Register, login, refresh, logout

Subtasks:

- Endpoints for login, refresh, logout, change and reset password.
- Hash passwords with argon2 or bcrypt; short-lived access token (15 min), refresh token (7 days) in an HttpOnly cookie.
- Store revoked refresh tokens in Redis until expiry.
- Login page in the front end with validation and error states.

Acceptance criteria:

- Valid credentials return tokens; invalid credentials return 401 without revealing which field was wrong.
- After logout, the old refresh token is rejected.
- Five failed logins in 10 minutes lock the account for 15 minutes (counter in Redis).

AUTH-2 Role-based access control and clinic scoping

Subtasks:

- Roles and permissions matrix; `@Roles()` decorator and guard.
- Every query filtered by `clinic_id` taken from the token, never from the request body.
- Front end route guards and hidden menu items per role.

Acceptance criteria:

- A receptionist calling an admin-only endpoint gets 403.
- A user of clinic A can never read or modify data of clinic B (proved by an automated test).
- Sidebar shows only the pages allowed for the role.

AUTH-3 Audit trail

Subtasks:

- Interceptor that records who, what, when, entity and before/after values for create, update and delete.
- Audit log screen (Settings) with filters by user, entity and date.

Acceptance criteria:

- Every appointment status change appears in the log within 1 second.
- Audit records cannot be edited or deleted through the API.

Epic 2 — Internationalisation: English, Arabic, French (Sprint 1–2)

I18N-1 Language switching and RTL

Subtasks:

- Configure i18next with namespaces per feature; language switcher in the top bar and user profile.
- Set `dir="rtl"` and `lang` on `<html>` for Arabic; use logical CSS properties (`margin-inline-start`, `ps-*`).
- Mirror icons that imply direction (arrows, chevrons); keep numbers, charts and phone numbers readable.
- Load an Arabic-capable font (Inter plus Noto Sans Arabic or IBM Plex Sans Arabic).

Acceptance criteria:

- Switching language updates the whole UI without a page reload and is remembered per user.
- In Arabic, the sidebar moves to the right and every screen is mirrored with no overlapping or clipped text.
- No hard-coded strings: a lint rule or script fails the build on missing translation keys.

I18N-2 Localised data and API messages

Subtasks:

- Format dates, times and numbers with `Intl` per locale; store all timestamps in UTC.
- Translate specialty names, appointment types, statuses and API error codes.
- Localised email and notification templates.

Acceptance criteria:

- The same appointment shows correct local date and time in all three languages.
- An API error shown to an Arabic user appears in Arabic, selected via the `Accept-Language` header.

7. Backlog, part 2: patients, doctors, appointments

Epic 3 — Patients (Sprint 2)

PAT-1 Patient directory API and screen

Subtasks:

- CRUD endpoints with search (name, ID, phone), filters (status, gender), sorting and pagination.
- Patient list: avatar initials, patient ID, phone, last visit, next appointment, status. Minimal sensitive data in the list.
- Soft delete (archive) instead of hard delete.

Acceptance criteria:

- Searching 10,000 seeded patients returns the first page in under 300 ms (indexed, cached for repeated queries).
- Duplicate detection warns when name plus date of birth or phone already exists.
- Archived patients disappear from lists but keep their appointment history.

PAT-2 Patient profile

Subtasks:

- Tabs: personal information, upcoming appointments, past visits, notes, contact preferences.
- "Schedule Appointment" button that opens the booking wizard with the patient preselected.
- Field-level permissions: doctors see clinical notes; receptionists see administrative data only.

Acceptance criteria:

- A receptionist cannot read or edit clinical notes (UI hidden and API returns 403).
- Profile data changes are written to the audit log.

Epic 4 — Doctors, staff and availability (Sprint 2–3)

DOC-1 Doctor and staff management

Subtasks:

- CRUD for doctors with specialty, default consultation duration and photo or initials avatar.
- Cards view and table view with availability status and utilisation percentage.

Acceptance criteria:

- Utilisation = booked minutes divided by available minutes for the selected day, verified by a unit test with fixed data.
- A doctor can be deactivated; future appointments are flagged for rescheduling, not silently deleted.

DOC-2 Working hours, breaks and leave

Subtasks:

- Weekly working hours with multiple intervals per day, breaks, public holidays and leave periods.
- Weekly schedule view on the doctor detail page.

- Slot generator: a function that returns free slots for a doctor and date.

Acceptance criteria:

- No slot is ever generated inside a break, a leave or outside working hours (unit tests with at least 15 cases, including midnight edge and daylight saving).
- Changing hours with existing bookings in the removed period shows a conflict list before saving.

Epic 5 — Appointments and calendar (Sprint 3–4)

APP-1 Appointment CRUD and lifecycle

Subtasks:

- Statuses: Scheduled, Confirmed, Checked In, In Progress, Completed, Cancelled, No-show; allowed transitions enforced in a state machine.
- Types: Consultation, Follow-up, Emergency, with default durations.
- Endpoints for list (filters, search, pagination), detail, create, update, cancel, bulk actions.

Acceptance criteria:

- An illegal transition (for example Completed to Scheduled) returns 409.
- Cancelling requires a reason and respects the cancellation policy from Settings.
- Appointment IDs are human-readable and unique (for example `AP-1001`).

APP-2 Conflict detection and slot locking

Subtasks:

- Database exclusion constraint (or transaction with row lock) preventing overlapping appointments per doctor.
- Redis lock `slot:{doctor}:{start}` with 5-minute TTL while a user is in the wizard.
- Return up to 3 alternative slots (same doctor nearest in time, or another doctor of the same specialty).

Acceptance criteria:

- Two simultaneous requests for the same slot: one succeeds, the other receives 409 with alternatives (proved by a concurrency test with 50 parallel requests).
- Abandoning the wizard releases the lock automatically.

APP-3 Create Appointment wizard (React)

Subtasks:

- Five steps: patient, specialty and doctor, date and time, details, review.

- Inline new-patient form; availability badges; morning, afternoon, evening slot filter; disabled unavailable slots.
- Conflict state with alternative slots; success screen with "View appointment" and "Book another".
- Draft kept if the user closes and reopens within the session.

Acceptance criteria:

- A user can complete a booking by keyboard only and on a 375 px wide screen.
- Each step validates before "Next"; "Back" never loses entered data.
- After confirmation, the appointment appears in the list and calendar without a manual refresh.

APP-4 Appointments list and calendar views

Subtasks:

- Table with sorting, filters (doctor, specialty, status, type), date range, bulk select, row actions menu.
- List, Day, Week, Month views; colour per doctor or type, never colour alone (status icon and text).
- Loading skeletons, empty and error states.

Acceptance criteria:

- Filters are reflected in the URL so a link can be shared.
- Week view renders 500 appointments without visible lag (under 100 ms interaction).
- On mobile the table becomes a card list.

APP-5 Reminders and confirmations

Subtasks:

- BullMQ delayed jobs at configurable offsets (for example 24 h and 2 h before).
- Provider interface with a mock email/SMS implementation; record delivery status.

Acceptance criteria:

- Rescheduling or cancelling removes the old reminder jobs.
- The UI shows a message as "Sent" only after the provider reports success; failures show "Failed" with a retry.

8. Backlog, part 3: smart scheduling and queue

Epic 6 — Smart Scheduling and optimisation (Sprint 4–5)

This is the signature feature. The optimiser is a **deterministic heuristic**, not machine learning. Its output is always a *suggestion* that staff preview and confirm; nothing changes until "Apply Schedule" is pressed.

SMT-1 Schedule metrics service

Subtasks:

- For a date and clinic compute per doctor: booked minutes, free minutes, idle gaps, utilisation, estimated waiting time (documented formula in `docs/`).
- Cache results in Redis (TTL 60 s) and invalidate on any appointment change.

Acceptance criteria:

- Metrics match a hand-calculated example in a unit test.
- Cached and uncached calls return identical values; cache is invalidated within 1 second of a booking.

SMT-2 Optimisation engine

Subtasks:

- Objectives: Minimise Waiting Time, Maximise Doctor Utilisation, Reduce Schedule Gaps.
- Algorithm (suggested): greedy or local search that moves flexible appointments (not Emergency, not locked) to better slots, respecting working hours, breaks, durations and specialty.
- Return a proposal: list of moves (from, to), metrics before and after, conflicts it could not resolve.
- Compute in a BullMQ job for large days; expose status polling or a WebSocket event.

Acceptance criteria:

- A proposal never creates an overlap, never moves an appointment outside the doctor's hours, and never moves Emergency or Checked-In appointments (property-based tests).
- On the seeded benchmark day, the "Minimise Waiting Time" objective lowers estimated average waiting by a measurable amount, or returns "No improvement found".
- Calling the engine twice with the same input returns the same proposal (deterministic).

SMT-3 Apply, preview and rollback

Subtasks:

- Endpoint to preview a proposal and endpoint to apply it atomically in one database transaction.
- Reject a proposal if the schedule changed since it was generated (version check).
- Undo the last applied optimisation within 15 minutes; write every move to the audit log.

Acceptance criteria:

- If any single move fails, no move is applied.
- A stale proposal returns 409 and the UI asks the user to regenerate.
- Affected patients and doctors receive a notification of the change.

SMT-4 Smart Scheduling screen

Subtasks:

- Control panel: date, objective selector, KPIs (availability, demand, utilisation, estimated waiting).
- Grid with doctors as rows, time slots as columns, appointment blocks coloured by type, breaks and unavailable periods hatched, conflict markers.
- Drag and drop to move a block manually with live validation.
- Right panel "Optimization Insights": suggested moves, expected waiting reduction, idle doctors, conflicts, alternative slots.
- States: successful optimisation, conflicts, no alternative slots, unsaved changes warning, results awaiting confirmation.
- Buttons "Preview Changes" and "Apply Schedule".

Acceptance criteria:

- Dropping a block on an invalid slot snaps back and explains why.
- Leaving the page with unsaved changes asks for confirmation.
- Each of the five states is reachable and has a screenshot in `docs/`.
- Works on tablet with a horizontally scrollable grid and a simplified list on mobile.

Epic 7 — Waiting list and patient queue (Sprint 5)

QUE-1 Queue engine

Subtasks:

- Check in an appointment to create a queue entry with number, priority (Normal, Urgent, Emergency) and check-in time.

- Actions: call next, start consultation, complete, reschedule, notify delay, no-show.
- "Call next" rule: highest priority first, then earliest appointment time, then earliest check-in.
- Keep the live queue in Redis (sorted set) and persist state changes in PostgreSQL.

Acceptance criteria:

- "Call next" picks the correct patient in a table of 10 unit-test scenarios.
- Two receptionists clicking "Call next" at the same time never call the same patient.
- Waiting duration is computed server-side and survives a page reload.

QUE-2 Queue screen and public display

Subtasks:

- Staff screen: summary cards (waiting, average wait, longest wait, checked in today), queue table, row actions.
- Highlight patients who exceed the target waiting time (configurable in Settings) using colour, icon and text.
- Public waiting-room screen at `/display/:clinicId` showing only queue number and room, with no names.
- Live updates through Socket.IO, with automatic reconnection.

Acceptance criteria:

- A check-in on one browser appears on every other open browser within 1 second.
- The public screen contains no personal data (verified by inspecting its API response).
- If the socket disconnects, a banner appears and data is re-fetched on reconnect.

9. Backlog, part 4: realtime chat and notifications, analytics, settings

Epic 8 — Realtime service: chat and notifications on MongoDB (Sprint 5–6)

RT-1 Realtime service setup

Subtasks:

- Second NestJS app with a Socket.IO gateway, Mongoose, Redis adapter, JWT validation on handshake (same secret or public key as the API).
- Rooms per clinic, per user and per conversation.
- Domain events from the API (`appointment.created`, `.cancelled`, `.rescheduled`, `queue.updated`) delivered over Redis pub/sub or BullMQ.

Acceptance criteria:

- A socket without a valid token is refused.
- Two instances of the service behind the Redis adapter both deliver events to connected clients.
- Users never receive events of another clinic (tested).

RT-2 Staff chat

Subtasks:

- Mongo collections `conversations` and `messages` (indexes on `conversationId, createdAt`).
- Direct and group chats between staff of one clinic; typing indicator, delivered and read receipts, online presence (Redis).
- Cursor-based message history (infinite scroll upwards); attachment links limited to images and PDF under 5 MB.
- Chat UI as a drawer or page with unread counters.

Acceptance criteria:

- A message appears for the recipient in under 500 ms and is persisted before being acknowledged.
- History loads 30 messages per page, ordered correctly, with no duplicates.
- Messages are escaped to prevent XSS; a user can read only conversations they belong to.
- Arabic text renders right-to-left inside the message bubble.

RT-3 Notification center

Subtasks:

- Mongo `notifications` collection: type, title and body keys (translated at read time), entity reference, `readAt`, `createdAt`, TTL index (90 days).
- Categories: reminders, confirmations, cancellations, schedule conflicts, delayed consultations, availability changes.
- Bell with unread counter, notification page with filters, actions "View Appointment" and "Mark as Read", mark all as read.
- Reminder settings page: channels (in-app, email, SMS) and timing per user.

Acceptance criteria:

- A new appointment produces exactly one notification per interested user (idempotent on retries).
- Unread counter stays correct across several open tabs.

- Delivery state is stored; "sent" is displayed only after a confirmed delivery.

Epic 9 — Dashboard and analytics (Sprint 3 for dashboard, Sprint 6 for analytics)

ANA-1 Executive dashboard

Subtasks:

- Greeting with the user's name, four KPI cards (today's appointments with change vs yesterday, patients waiting, available doctors, average waiting time vs last week).
- Today's schedule timeline with current-time marker and next appointment highlighted; clinic activity feed.
- Four charts: weekly volume, status distribution, doctor utilisation, waiting-time trend.
- Aggregations on the server; responses cached in Redis for 60 s.

Acceptance criteria:

- KPI numbers equal the result of a manual SQL check on the seeded data.
- Dashboard first paint under 2 s on seeded data; skeletons shown while loading.
- Matches the Figma layout at 1440 px width and stacks correctly on tablet and mobile.

ANA-2 Analytics and reports

Subtasks:

- Charts: volume over time, no-show and cancellation rates, waiting time, utilisation, distribution by specialty, peak booking hours, average duration, throughput.
- Date-range picker with comparison against the previous period.
- Export to CSV (streamed) and PDF (server-rendered).

Acceptance criteria:

- Changing the range updates all charts and the comparison percentages correctly.
- A CSV for 12 months opens in Excel with correct Arabic and French characters (UTF-8 with BOM).
- Charts have legends, tooltips, readable axes and an empty state when no data exists.

Epic 10 — Settings and clinic configuration (Sprint 6)

SET-1 Settings screens

Subtasks:

- Sections: clinic information, departments and specialties, working hours, appointment durations, breaks and holidays, cancellation policy, reminder preferences, user roles and permissions, audit log, appearance and accessibility.

- Forms with inline validation, unsaved-changes indicator, confirm dialogs for destructive actions, save and cancel.
- Settings cached in Redis; cache cleared on update.

Acceptance criteria:

- Invalid input shows a field-level message in the user's language and blocks saving.
- A change of cancellation policy applies immediately to new cancellations and is recorded in the audit log.
- Only Clinic Admin and Super Admin can open these screens.

10. Backlog, part 5: design system, testing and DevOps

Epic 11 — Front-end shell and design system (Sprint 1–2, continuous)

UI-1 Design tokens and component library

Subtasks:

- Tailwind theme (or MUI theme) with the colour tokens in section 1, Inter font, 10 to 16 px radii, subtle shadows.
- Components: buttons (primary, secondary, tertiary, destructive, icon), inputs, searchable select, date and time pickers, tabs, dropdown, tooltip, popover, data table (sort, pagination), KPI card, status badge, toast, confirm dialog, skeleton, empty state, error state.
- Lucide icons only. Document components in Storybook.

Acceptance criteria:

- Every component has a Storybook story with its states (default, hover, focus, disabled, error).
- Text contrast meets WCAG AA (4.5:1); interactive targets are at least 40 px.
- Status badges show an icon or text and never rely on colour alone.

UI-2 Application shell

Subtasks:

- Sidebar (full and collapsed), top bar with breadcrumbs, global search, clinic selector, notification bell, help, user menu, "+ New Appointment" button.
- Responsive behaviour: full sidebar on desktop, collapsible on tablet, bottom or drawer navigation on mobile.
- Global error boundary, 404 page, offline banner.

Acceptance criteria:

- The shell visually matches the mockups at 1440 px; the active item is clearly highlighted.
- Sidebar state is remembered; keyboard users can reach every item and see focus.
- The shell is usable at 375, 768 and 1440 px in all three languages.

Epic 12 — Quality, security and DevOps (continuous, closing in Sprint 6)

QA-1 Automated testing

Subtasks:

- Unit tests (slot generator, state machine, optimiser, queue ordering).
- Integration tests with Supertest on a test database (Testcontainers or Compose).
- End-to-end tests with Playwright for the three journeys in section 12.

Acceptance criteria:

- Back-end line coverage at least 70% overall and at least 90% on slot generation, conflict detection, optimiser and queue modules.
- All three E2E journeys pass in CI in under 10 minutes.

QA-2 Security hardening

Subtasks:

- Helmet, CORS allow-list, rate limiting (Redis), request size limits, parameterised queries only.
- Secrets from environment variables; dependency audit in CI.
- Review against the OWASP Top 10 and write a one-page threat model.

Acceptance criteria:

- `npm audit` reports no high or critical vulnerabilities at release.
- Login and booking endpoints are rate limited (verified by a test).
- No personal data appears in application logs.

QA-3 Performance and observability

Subtasks:

- Add database indexes based on `EXPLAIN` output; verify cache hit ratios.
- Load test (k6 or Artillery): 100 concurrent users on booking and dashboard endpoints.
- Structured logs with request IDs.

Acceptance criteria:

- p95 response time under 400 ms for list endpoints on seeded data under the load test.

- A request can be traced across the API and realtime service by one request ID.

QA-4 Delivery and documentation

Subtasks:

- Production Docker images (multi-stage), CD pipeline to a free-tier host or a VPS.
- Documentation: README, architecture diagram, ERD, API (Swagger), ADRs for key decisions, user manual with screenshots in the three languages.
- Demo script and seeded demo accounts for every role.

Acceptance criteria:

- A person who has never seen the project can run it locally by following the README only.
- The deployed demo is reachable by URL and logins for each role work.

11. Non-functional requirements

Area	Requirement	How it is checked
Performance	p95 under 400 ms for list endpoints; dashboard under 2 s	k6 load test
Security	JWT with refresh rotation, RBAC, clinic isolation, rate limiting, audit log	Integration tests, review
Privacy	Fictional data only; minimal personal data in lists and public screen; no personal data in logs	Code review, log inspection
Accessibility	WCAG 2.1 AA, keyboard navigation, focus states, status not by colour alone	axe checks, manual keyboard test
Internationalisation	EN, AR (RTL), FR across all screens and API messages	Visual review, missing-key check in CI
Responsiveness	Desktop 1440 px, tablet 768 px, mobile 375 px; card lists instead of wide tables on mobile	Playwright screenshots
Reliability	Atomic schedule changes, idempotent notifications, socket auto-reconnect	Failure tests

Area	Requirement	How it is checked
Maintainability	TypeScript strict mode, linting, module boundaries, ADRs	CI, review

12. Milestones, journeys, deliverables and grading

Sprint plan (6 sprints of 2 weeks)

Sprint	Goal	Demo at the end
1	FND-1 to 3, AUTH-1, I18N-1, UI-1, UI-2	Login, empty shell in 3 languages, CI green
2	AUTH-2, AUTH-3, I18N-2, PAT-1, PAT-2, DOC-1	Patients and doctors with roles and audit
3	DOC-2, APP-1, APP-2, ANA-1	Slots, conflict-safe booking API, dashboard
4	APP-3, APP-4, APP-5, SMT-1	Full booking wizard, list and calendar, reminders
5	SMT-2 to SMT-4, QUE-1, QUE-2, RT-1	Smart Scheduling screen, live queue
6	RT-2, RT-3, ANA-2, SET-1, QA-1 to QA-4	Chat, notifications, analytics, settings, deployment

Three end-to-end journeys that must work in the final demo (also your E2E tests)

1. Dashboard, Create Appointment, select patient, choose doctor, select an available slot, review, confirm, appointment details.
2. Appointments, a booking conflict is detected, view alternative slots, preview optimisation, apply schedule.
3. Waiting list, check in a patient, call next patient, start consultation, complete the visit.

Deliverables

- Source code in a Git repository with a clean history and pull requests.
- Running system via `docker compose up` and a deployed demo URL.
- Documentation: README, ERD, architecture diagram, Swagger, ADRs, user manual (EN, AR, FR screenshots).
- Test report with coverage and the load test result.
- A 15-minute live demo and a 10-minute technical defence; each member answers questions on their own code.

Grading (100 points)

Criterion	Points
Functional completeness against acceptance criteria	25
Back-end architecture, data modelling and correct use of PostgreSQL, Redis and MongoDB	20
Smart scheduling and queue quality (correctness, tests, UX states)	15
Front-end quality: fidelity to the design, responsiveness, accessibility, i18n and RTL	15
Realtime features (chat, notifications, live queue)	10
Testing, security and CI/CD	10
Documentation, demo and teamwork (individual contribution from Git history)	5

Rules: all work is original and versioned in Git; copied code must be credited and understood; AI assistants may be used, but each student must be able to explain and modify every line they submit. Late submission loses 10% per day.